ToolX EdgeSync

Team Name: XRunners

Yuvraj Dhadwal, Patrick Huynh, Minjae Lee, Arthur Mahu, Christian Vela

Client: Yassine Bennani, Alha Kane

Repository: https://github.com/yuvrajdhadwal/ToolX-EdgeSync-DCA-6101

Table of Contents

Table of Figures

Terminology

API: an Application Programming Interface (API) is a defined set of rules that allow different software systems to communicate with each other.

Fast API: a modern Python web framework used to quickly build high-performance REST APIs with automatic documentation support.

OTA: Over-The-Air updates are a method of remotely delivering software or firmware updates to devices via a network connection.

SQLite: a lightweight, file-based relational database system that stores data locally without requiring a separate server.

Linux: a free and open-source operating system kernel

Edge Devices: hardware components that are entry points to a network. They process data near the source before sending data packets to central servers or cloud systems for further computation.

Cloud-Native: a system that takes advantage of cloud resources and development operations. This is beneficial since cloud services can be scaled from 0 to 100 with minimal effort from the developer.

React.ts: a TypeScript file used in React applications that combines React's component-based UI library with TypeScript's static typing for safer and more scalable frontend development.

JWT: JSON Web Token is a compact, URL-safe token format used to securely transmit verified user identity and claims between parties, typically for authentication and authorization.

TLS: Transport Layer Security is a cryptographic protocol that encrypts data in transit to provide secure communication over a network, such as HTTPS.

Azure IoT: Infrastructure that provides the connectivity, processing, and management capabilities needed to integrate physical devices with cloud and edge services

Azure BLOB Storage: an object storage solution for the cloud, designed to store massive amounts of unstructured data such as text, images, videos, backups, and logs. It is highly scalable, durable, and accessible.

Node.js: an open-source, cross-platform JavaScript runtime environment that allows developers to run JavaScript on the server side.

Introduction

Background

We will design and implement a simplistic yet scalable software infrastructure for managing over-the-air (OTA) firmware updates across industrial edge devices running embedded Linux. The system will streamline update deployment, ensure end-to-end security, and support future growth with modular architecture and cloud-native components.

Document Summary

The System Architecture section describes the static components of our system's backend and frontend. The section includes the rationale behind the decisions of architectural design that the team made. The static system architecture diagram displays architectural components and logical relationships between each component.

The Data Design section describes the database structure of our application. We provide an Enhanced Entity Relationship Diagram for the SQLite database. Additionally, this section will also cover subsections for firmware file usage, data exchanges between edge devices and the IoT Hub, as well as for any security considerations involved.

Static System Architecture

Introduction

With this system we hope to establish these core functionalities: The workflow for the acceptance and declining of firmware, developed by the developers, by the developer manager, the workflow of deployment of firmware that comprises of the business manager approving firmware sent from the developer manager, the workflow of the field technician either accepting or denying the firmware and the installation and tracking of the firmware's history and other metadata.

Rationale

For our team's security considerations, we decided to use JWT authentication within our FastAPI backend because JWTs are stateless, meaning the client server can verify the user's identity without having to store their session data into our backend. JWT uses asymmetric signing using public/private key pairs to ensure the integrity of the token, allowing the client to be able to verify the sender's identity and ensure that the data isn't altered in the backend. Additionally, JWTs use short-lived expiration tokens to mitigate token theft. Asides from JWTs, we will also need to implement network security measures such as Transport Layer Security (TLS) to ensure data communication over a network is secured and encrypted.

Static Architecture

All users interact with our system through the frontend layer. At this layer, they can view all the possible firmware versions, devices, and any information they need for their role. The frontend is implemented with Typescript and Node.js. Field/Shop Professionals will also have the functionality of interacting with a device shell. From this shell, they can communicate to the system that their device is ready to install a deployed firmware. These devices have a C++ state machine that is able to handle all the logic necessary for our application. The Webapp requests and posts data to a backend API backed by Python FastAPI. This API handles all the business logic for the webapp, including storing persistent data in a SQLite Database: including but not limited to device location, current device version, and different kinds of firmware. To store the actual binary firmware that is to be installed on devices, we elected to use Azure BLOB Storage since it is designed to store large files easily while SQLite is meant to be lightweight. To handle the orchestration between this Python backend and thousands of devices globally, we utilize Azure IoT. Our Azure IoT handles communication and networking between the devices, Azure BLOB, and the Python backend. This is illustrated in Figure 1.

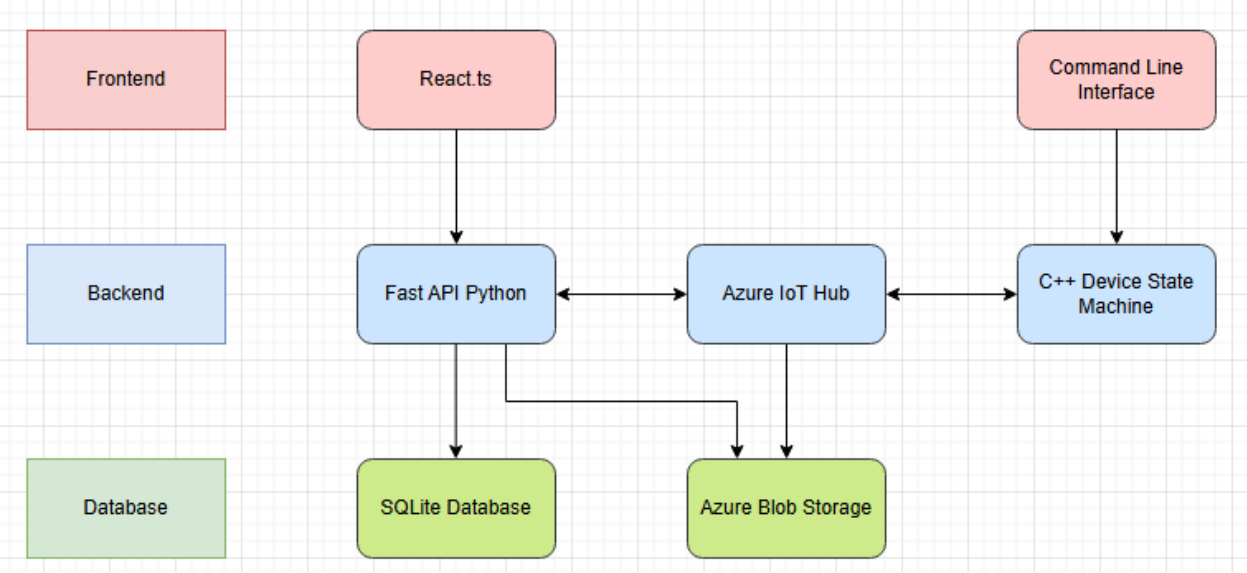

Figure 1 UML of Static System Architecture

Data Design

Introduction

Figure 2 is an Enhanced Entity Relationship diagram representing our application. The diagram shows the types of entities, attributes, and relationships between entities. The type of database of the project is SQLite. After the diagram, file usage, data exchange, and data security considerations are discussed.

Enhanced Entity Relationship Diagram

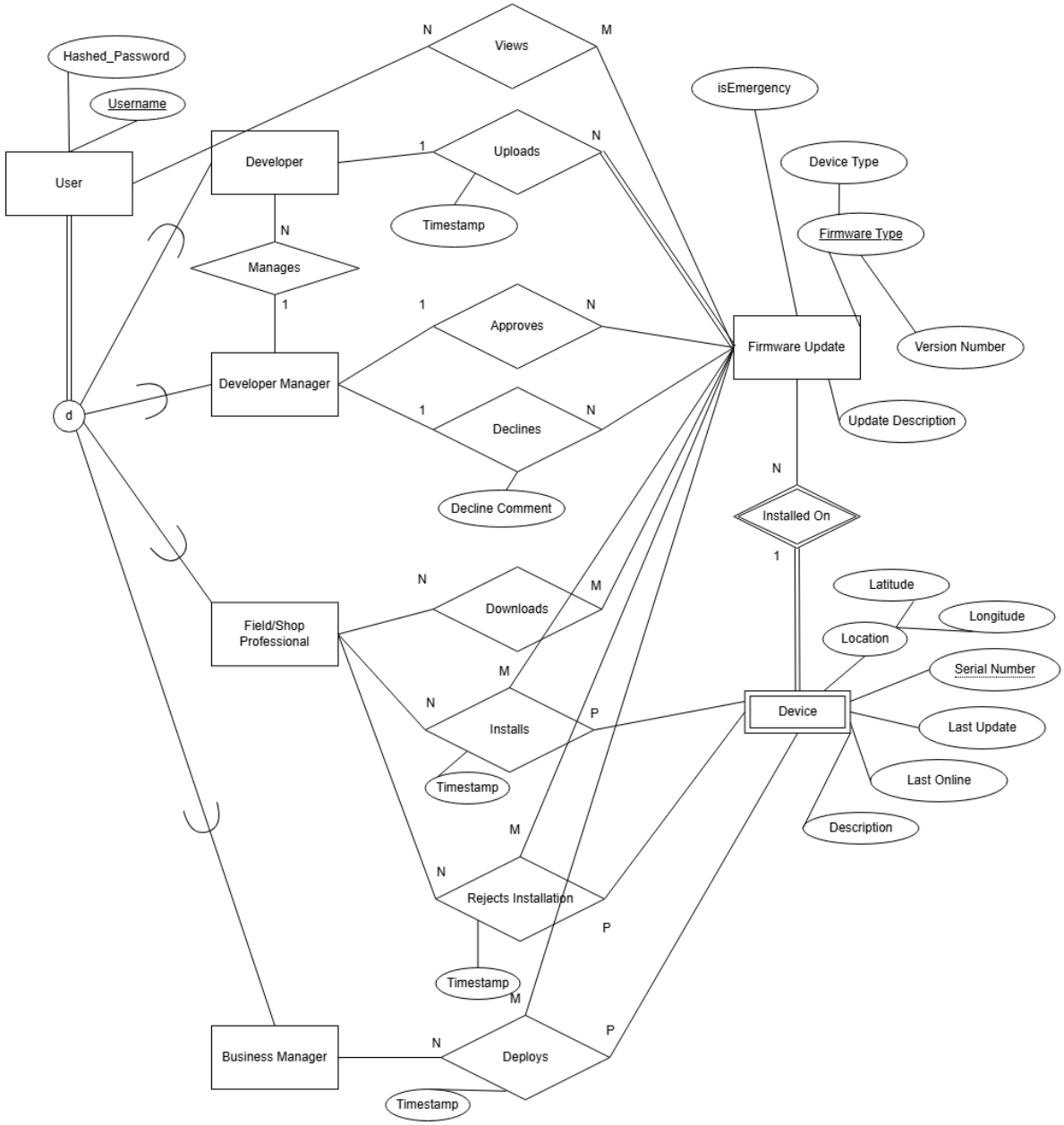

Figure 2 Entity Relationship Diagram

Here is a list of all major entities:

1. Developer: One of the roles of the user, who can upload firmware updates. The developer is managed by the developer manager.
2. Developer Manager: One of the roles of the user, who can approve or decline firmware updates and manage developers.

3. Field/Shop Professional: One of the roles of the user, who can download firmware updates to local devices or install the firmware update to the device or reject firmware update installation.
4. Business Manager: One of the roles of the user, who can deploy the firmware updates to the devices.
5. Firmware Updates: Contains the information about firmware type, update description from the developer, and a Boolean flag indicating if the update is an emergency or not. Firmware updates are installed on devices.
6. Devices: Contains the location of the devices, timestamps of last update and last online, and the description of the devices.

File Use

Uploaded firmware will be stored as compiled, binary files zipped together into a zip file. These files will be stored in Azure Blob Storage.

Data Exchange

Basic Transport Layer Security (TLS) will be utilized in communication between Azure IoT Hub and Edge Devices. Specifically, we will be employing the X.509 to format key certificates for our TLS.

Security Considerations

Malicious users can hijack our network packets and try to inject their own binary files into our channels. This is why we need basic protection. Furthermore, data encryption is needed for the log in information as well as the database data. TLS will be used to ensure secure connections when exchanging data across channels. User Login is required for this application. Users will login by entering a username and password. Passwords are hashed in database for protection. We do not work with any financial data on this project, and the only PII in our application would be the names of the developers of certain firmware. This keeps the information safe since the only other users that may see the developer's name would be the developer manager who is already in charge of the developer. The biggest privacy concern at the moment is ensuring that user login is not only being hashed in our database, but this login data should also be salted on top of being hashed to ensure that password multiple of the same password do not provide hash when being stored in our database.